Masked Attention Pooling

Padding and permutation invariance in Lean

Local project guide | Validated October 9, 2026

1 The operation we are proving correct

In multiple instance learning (MIL), a bag contains feature vectors for a variable number of instances. A batch often stores those vectors in a larger array with unused rows. Attention pooling should depend on the real instances, regardless of how much padding was added or how the instances were ordered.

Let I be a finite set of row indices, d the number of features, and

$$x_i \in \mathbb{R}^d, \qquad s_i \in \mathbb{R}, \qquad m_i \in \{0, 1\}.$$

Here x_i is an instance's feature vector, s_i its attention logit, and m_i its validity mask. For a prefix of length ℓ stored in n rows, $m_i = 1$ exactly when $0 \le i < \ell$. The proofs allow any mask, including invalid rows between valid ones.

Define the attention mass and the safely masked features by

$$w_i = \begin{cases} e^{s_i}, & m_i = 1, \\ 0, & m_i = 0, \end{cases} \qquad v_i = \begin{cases} x_i, & m_i = 1, \\ 0, & m_i = 0. \end{cases}$$

The normalizer and the pooled vector are

$$Z = \sum_{i \in I} w_i, \qquad D(Z) = \begin{cases} Z, & Z > 0, \\ 1, & Z \le 0, \end{cases} \qquad \boxed{\operatorname{Pool}(x, s, m) = \frac{\sum_{i \in I} w_i v_i}{D(Z)}}. \tag{1}$$

All sums and vector operations are componentwise. If at least one row is valid, $e^{s_i} > 0$ gives $Z > 0$, and the valid weights w_i/Z sum to one. If every row is invalid, both sums vanish and the output is the zero vector. This also covers a bag with no rows.

Why masking features alone is insufficient

Take two scalar instances $x_1 = 2$, $x_2 = 6$, both with logit zero. The correct output is $(2+6)/2 = 4$. If we add a zero-feature padded row but leave it inside softmax, its mass is $e^0 = 1$, giving $(2+6+0)/3 = 8/3$ instead. The padded row must contribute zero to *both* the numerator and normalizer.

2 The two invariance proofs

(a) Adding padding leaves the output unchanged

Let P be any finite set of added rows, disjoint from I. Give those rows arbitrary real feature vectors and logits, but set their masks to zero. Write the extended arrays as x', s', m'. On original rows they equal x, s, m; on rows in P, $m'_p = 0$, so $w'_p = 0$ and $v'_p = 0$.

Splitting the sum over the disjoint union gives

$$Z' = \sum_{r \in I \sqcup P} w'_r = \sum_{i \in I} w_i + \sum_{p \in P} 0 = Z,$$
$$N' = \sum_{r \in I \sqcup P} w'_r v'_r = \sum_{i \in I} w_i v_i + \sum_{p \in P} 0 = \sum_{i \in I} w_i v_i = N.$$

Since the normalizer is unchanged, its guarded denominator is unchanged too:

$$\boxed{\mathrm{Pool}(x', s', m') = \frac{N'}{D(Z')} = \frac{N}{D(Z)} = \mathrm{Pool}(x, s, m).}$$

The argument does not require a nonempty bag. It also does not require the padded features or logits to be zero; only their masks must be zero.

(b) Permuting paired rows leaves the output unchanged

Let $\pi : I \to I$ be a bijection. Move each instance together with its logit and mask:

$$x'_i = x_{\pi(i)}, \qquad s'_i = s_{\pi(i)}, \qquad m'_i = m_{\pi(i)}.$$

Then $w'_i = w_{\pi(i)}$ and $v'_i = v_{\pi(i)}$. A bijection visits every original row exactly once, so reindexing each finite sum gives

$$Z' = \sum_{i \in I} w_{\pi(i)} = \sum_{r \in I} w_r = Z,$$
$$N' = \sum_{i \in I} w_{\pi(i)} v_{\pi(i)} = \sum_{r \in I} w_r v_r = N.$$

Consequently,

$$\boxed{\mathrm{Pool}(x \circ \pi, s \circ \pi, m \circ \pi) = \mathrm{Pool}(x, s, m).}$$

A permutation of only the real instances is a special case: extend it to the whole array by fixing the padded positions. A more general permutation can move padded positions too, provided the mask moves with them.

When logits are computed from the instances

A shared instance-local scorer $f : \mathbb{R}^d \to \mathbb{R}$ satisfies $s_i = f(x_i)$. After shuffling, $f(x'_i) = f(x_{\pi(i)}) = s_{\pi(i)}$, exactly as required above. This includes a shared linear layer, MLP, or gated scorer. A scorer that depends on position or an order-dependent encoder requires an additional argument.

3 How Lean expresses the proof

The general proofs are in `MaskedAttention/Basic.lean`. They quantify over arbitrary finite row sets and feature dimensions, rather than testing particular feature values.

Lean notation	Meaning
`Fin d`	A bounded feature index: $0, \ldots, d-1$.
`[Fintype I]`	The row index type is finite, so we can sum over it.
`values`	A function taking a row index and a feature index to a real value.
$e : K \simeq I$	A bijection from a new row index set to the original one.
`funext j`	Prove two output vectors equal by proving equality at each feature j.
`e.sum_comp`	Reindexing a finite sum by a bijection preserves its value.

Permutation: the same reindexing identity twice

The theorem `pool_reindex` uses the identity

$$\sum_{k \in K} g(e(k)) = \sum_{i \in I} g(i).$$

It first applies this identity to $g(i) = w_i$ and then to $g(i) = w_i v_{i,j}$. Once both sums are rewritten, the two sides of the pooling equality are identical. The theorem permits different finite index types K and I as long as they are related by a bijection.

Padding: split the index set and simplify the zero terms

Lean represents the padded row set by a disjoint union, `Sum I P`. `Sum.elim` selects the original data on one side and padded data on the other. The padding mask is always false. The actual checked proof body is

```
funext j
simp [pool, mass, Fintype.sum_sum_type]
rfl
```

Here `Fintype.sum_sum_type` splits the union sum into two sums. `simp` unfolds the definitions and removes zero terms; `rfl` finishes the remaining definitional equality. The short script is backed by general finite-sum results from mathlib.

Additional checked properties

`pool_masked_congr` proves that changing invalid features or logits has no effect. `pool_all_masked` proves the zero-output convention. `attention_permutation` applies the permutation theorem to an arbitrary shared instance-local scorer. These results use the same pooling definition as Equation (1).

4 From the JAX function to the certified theorem

The certified JAX kernel in `python/pooling.py` is

```
def masked_pool(values, logits, mask):
    safe_logits = jnp.where(mask, logits, 0.0)
    safe_values = jnp.where(mask[:, None], values, 0.0)
    weights = jnp.where(mask, jnp.exp(safe_logits), 0.0)
    total = jnp.sum(weights)
    denominator = jnp.where(total > 0.0, total, 1.0)
    return jnp.sum(weights[:, None] * safe_values,
                   axis=0) / denominator
```

`values` has shape $[n, d]$, while `logits` and `mask` have shape $[n]$. Safe selection suppresses invalid values before exponentiation and multiplication; an invalid logit is not allowed to create attention mass.

The connection is checked at each function boundary

1. JAX traces the kernel for the fixed shapes $[3, 2]$ and $[5, 2]$.
2. Sasha Rush's `jax-lean` library emits a readable Lean tensor function and a separately imported, typed mathematical Jaxpr for each shape.
3. Generated translation certificates prove that evaluating each imported Jaxpr agrees with its readable Lean function for every modeled real input.
4. `generated3_spec` and `generated5_spec` identify those functions with the general pooling definition. They check the mask branches, reductions, and guarded denominator.
5. The generic invariance theorems apply. The final statements `certified_permutation` and `certified_padding` carry these properties to the mathematical Jaxpr evaluator.

The concrete padding theorem compares the separately traced 3-row and 5-row kernels with two arbitrary added rows whose masks are false. The reusable padding theorem covers any finite number of added rows. Generated Jaxprs have fixed shapes; the general theorem does not make JAX shape-polymorphic.

What the certificate establishes

The proof is an exact identity in **ideal real arithmetic**. It does not certify floating-point rounding, XLA execution, or the correctness of JAX itself. JAX tracing and the Python importer remain trusted.

The runtime helper `stable_masked_pool` subtracts the maximum valid logit before exponentiation, avoiding overflow for large finite logits. That helper and the prefix-length/scorer wrapper `attention_pool` are numerically tested, but their own Jaxprs are not certified in this version. The real-valued theorem excludes NaN and infinity; runtime tests separately check that nonfinite values in invalid rows are suppressed.

5 Validation and using the project

The local check script completed successfully on macOS with a JAX CPU device. The environment uses JAX and jaxlib 0.8.0, Lean 4.33.0, and the matching mathlib release. The official Lean release archive's SHA-256 was verified before installation.

Check	Result
Lean compilation	All project proofs and generated certificates compile.
Theorem axiom audit	Seven audits use only `propext`, `Classical.choice`, and `Quot.sound`. No `sorryAx` or custom axioms.
JAX numerical tests	Six tests pass: reference values, padding, permutation, masked/empty bags, and numerical stability.
Generated-file check	Both generated files match the pinned source and library.

The three-instance demo uses

$$X = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 2 & -1 \end{pmatrix}, \qquad a = \begin{pmatrix} 0.2 \\ -0.1 \end{pmatrix}, \qquad s = Xa = \begin{pmatrix} 0 \\ 0.2 \\ 0.5 \end{pmatrix}.$$

With all three rows valid, the output is approximately

$$\mathrm{Pool}(X, s, m) \approx (2.057208,\ 1.3531581).$$

Adding two masked rows or shuffling the three real rows produces the same displayed output. In general, floating-point reordering can change the last few bits, so numerical tests use tolerances. Tests support execution; the Lean theorems establish the exact mathematical identities.

Run the demo or repeat all checks

```
cd ~/Developer/masked-attention-lean
uv run python python/demo.py
./scripts/check.sh
```

Read `MaskedAttention/Basic.lean` first, then `MaskedAttention/Connection.lean`. `validation.log` preserves the successful check output. `README.md` and `WALKTHROUGH.md` provide additional source-file guidance.

Sources and project boundary

Library: `srush/jax-lean`. Pinned revision:
`570aed69049f8e2931df9f55175313a340bcb456`.
Proof template: `examples/transformer/proofs/TransformerProofs.lean`.
Arithmetic model and trust boundary: `SUPPORTED_JAXPR.md`.

This repository contains the pooling example and a pinned upstream dependency. No upstream modifications or pull requests are included. The results concern pooling structure; they do not establish model accuracy or the invariance of an entire contextual encoder or training pipeline.